

Scene Graph Construction and Open-Vocabulary Mobile Manipulation on the HSR-C Robot: Methodology and Code Correlation

HSR-C OVMM Project

June 14, 2026

Abstract

This report documents how the project builds a 3D scene graph from a Gazebo simulation (or from a real RGB-D scan), how an LLM is used to cluster furniture into rooms and to reason about object locations, and how a single mission command drives the HSR-C robot through navigation, active-perception observation, open-vocabulary detection, and grasping. Every demo-day requirement is mapped to the exact files and functions that implement it. Particular attention is given to the scene-graph geometry pipeline: object positions, sizes, and centroids are derived entirely from Gazebo's own model description files (SDF collision geometry), with no hand-picked dimensions.

Contents

1	Introduction	3
1.1	Demo-Day Requirements	3
1.2	Scope of this document	3
2	Repository Layout	4
3	System Architecture	4
4	Scene Graph Construction Methodology	5
4.1	World file parsing and the semantic registry	6
4.2	Geometry extraction from Gazebo SDF	7
4.3	Point cloud synthesis	13
4.4	ObjectNode: centroid, pose, and dimensions	13
4.5	SceneGraph: connections and persistence	14
4.6	End-to-end build flow	15
4.7	Resulting scene graph	16
4.8	The real-scan pipeline (Mask3D)	16
5	Demo-Day Requirement to Code Correlation	17
5.1	Requirement 2: room clustering with an LLM	18
5.2	Requirement 3: Scenario A / B target resolution	19
5.3	Requirement 4: navigating to the candidate location	20
5.4	Requirement 5: observing the location and detecting the object	20
5.5	Requirement 6: detected – point cloud, grasp, arm motion	20
5.6	Requirement 7: not detected – next location, then LLM fallback	21
5.7	Requirement 8: active perception (head control)	22

5.8	Requirement 10: a single command for steps 3–7	23
6	Single-Command Mission: Detailed Flow	23
7	Running the Pipeline	26
7.1	One-time setup	26
7.2	Bring up the simulation	26
7.3	Build the scene graph and room clusters (once per scene)	26
7.4	Run the mission (requirements 3–7, single command)	26
8	Conclusion	27

1 Introduction

1.1 Demo-Day Requirements

The target capability set for the demo is:

- (1) From the ground truth of the Gazebo simulator, create a 3D scene graph programmatically.
- (2) Use an LLM API to cluster furniture into rooms.
- (3) For a given object to search for, support two scenarios: **Scenario A** – the object is *not* in the scene graph, so the LLM is queried for the object and predicts the top- k candidate locations; **Scenario B** – the object *is* already in the scene graph, so its known location is used directly.
- (4) The robot navigates to the first candidate location (either the LLM’s top prediction, or the scene-graph location).
- (5) The robot observes the location and either detects or does not detect the object.
- (6) If detected: build an RGB-D point cloud of the object, plan a grasp, and move the arm.
- (7) If not detected: move to the next predicted location; in Scenario B, once the known location fails, fall back to querying the LLM for predictions.
- (8) Demonstrate active perception: control of the robot’s head to look at candidate locations.
- (9) Support the same scene-graph construction from a *real* RGB-D scan, using Mask3D segmentation.
- (10) Steps 3–7 must be runnable as a single command.

1.2 Scope of this document

Section 2 gives a repository map. Section 3 shows the overall system architecture. Section 4 is a deep dive into the scene-graph construction pipeline, including the **geometry-extraction module** that reads object dimensions and centroid offsets directly from Gazebo’s SDF model files (no hardcoded sizes anywhere). Section 5 walks through each demo-day requirement and points to the exact code that implements it. Section 6 gives detailed flowcharts of the single-command mission. Section 7 is a quick command reference.

2 Repository Layout

Path	Purpose
core/perception/scene_graph/	SceneGraph, ObjectNode, the Gazebo and Mask3D scene-graph builders, and the new gazebo_geometry module
core/perception/segmentation/	Mask3D output loader (mask3d_interface.py)
core/perception/detection/	SAM3 REST client and mask→pointcloud lifting
core/perception/grasping/	Geometric top-down grasp planner
core/perception/	head_geometry.py – pure pan/tilt geometry for active perception
core/llm_zone/	DeepSeek-based room clustering, scene queries, and active-perception viewpoint geometry
core/missions/	search_and_fetch.py – the ROS-agnostic mission runner (demo steps 3–7)
ros2_ws/src/fetcher/	ROS 2 nodes: Nav2 goal execution, scene-graph builder entry point, HsrRobotAdapter, single-command mission node
ros2_ws/src/tmc_gazebo/	The apartment.world.xacro world file and all Gazebo model SDFs
data/scene_graph/	Generated graph.json, scene.json, rooms.json, objects/*.json

3 System Architecture

Figure 1 shows the full pipeline. Two parallel inputs – Gazebo’s ground-truth world description, and a real RGB-D scan processed with Mask3D – are converted into the same **SceneGraph** / **ObjectNode** representation and the same JSON files. From there, two DeepSeek-backed LLM modules (room clustering and scene query) and the single-command mission runner operate identically regardless of which pipeline produced the scene graph.

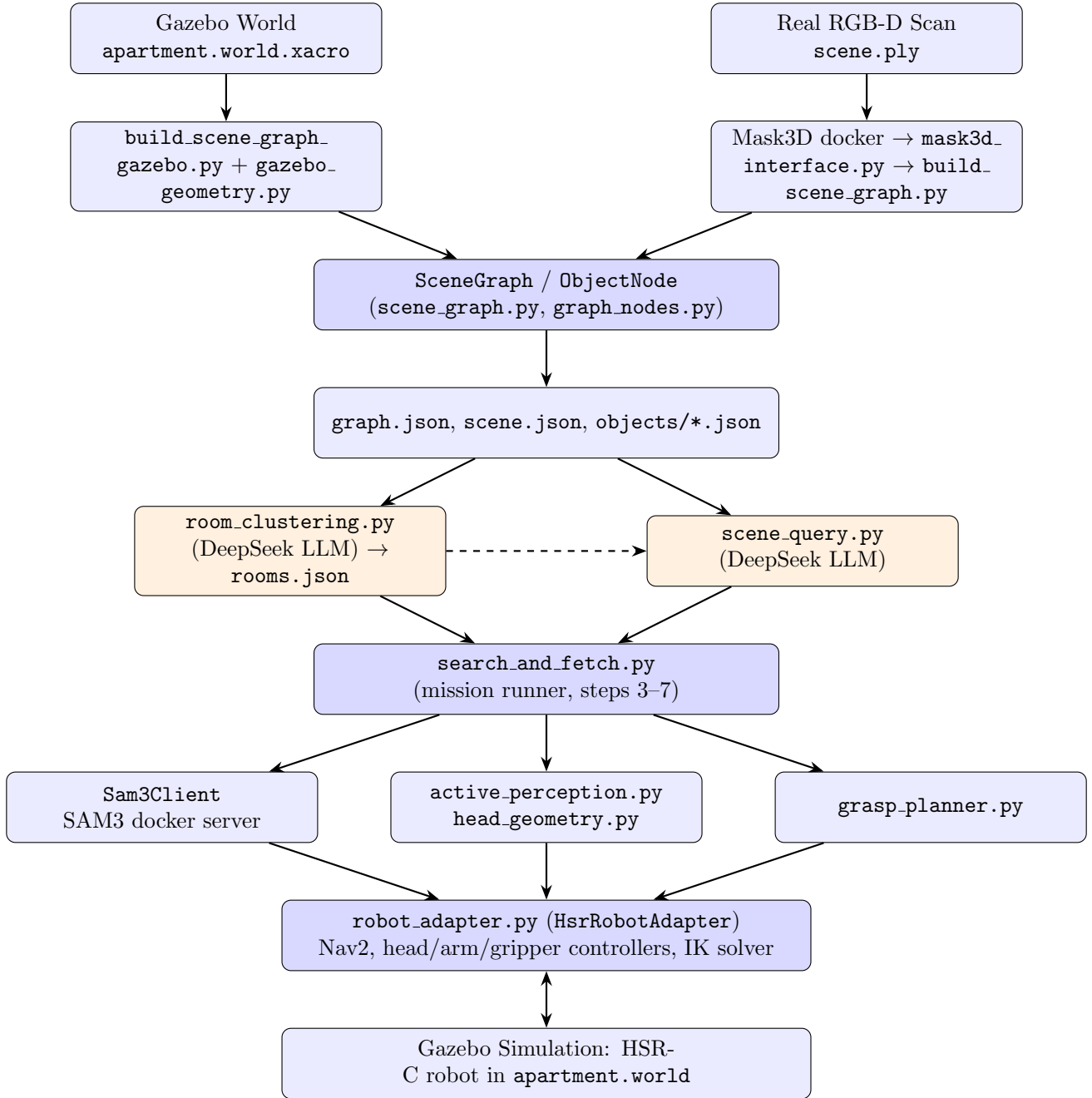


Figure 1: System architecture. Dashed arrow: room labels from `rooms.json` enrich the LLM’s location predictions in `scene_query.predict_locations`. The double-headed arrow between the robot adapter and Gazebo represents the bidirectional flow of commands (Nav2 goals, joint trajectories) and sensor data (camera, depth, TF, joint states).

4 Scene Graph Construction Methodology

This section explains, step by step, how `core/perception/scene_graph/build_scene_graph_gazebo.py` turns `apartment.world.xacro` into the `SceneGraph` object and its JSON outputs – and, in particular, how **every geometric quantity (object position, size, centroid offset)** is derived from **Gazebo’s own files**, with no hand-picked numbers.

4.1 World file parsing and the semantic registry

`apartment.world.xacro` is a Xacro file: a templated XML format that expands to plain SDF. `load_registry()` shells out to the xacro CLI and then walks the resulting `<world><include>` elements:

Listing 1: `build_scene_graph_gazebo.py`: world parsing (excerpt)

```
1 expanded = subprocess.run(
2     ["xacro", str(world_xacro)], capture_output=True, text=True, check=True
3 ).stdout
4 world = ET.fromstring(expanded).find("world")
5
6 registry = []
7 for include in world.findall("include"):
8     name = include.findtext("name")
9     label = NAME_TO_LABEL.get(name)
10    if label is None:
11        continue
12    pose = tuple(float(v) for v in include.findtext("pose").split())
13    dims, center = model_local_bbox(include.findtext("uri"))
14    registry.append({"name": name, "label": label, "pose": pose,
15                    "dims": dims, "center": center})
```

Each `<include>` in the expanded world file provides three pieces of **ground truth**, read directly from the simulation description:

- `<name>` – the instance name (e.g. `chair00`), used only to look up a semantic label;
- `<uri>` – a `model://<name>` reference to the object’s SDF model, used to derive its geometry (Section 4.2);
- `<pose>x y z roll pitch yaw</pose>` – the object’s real pose in the world frame.

The only remaining hand-written table. `NAME_TO_LABEL` maps each `<include><name>` to a human-readable semantic category (Table 1). This is *semantic* configuration – it tells the system “chair00 is a chair” – exactly analogous to the ScanNet200 label table (`core/utils/scannet_200_labels.py`) used by the real-scan pipeline, or the `IMMOVABLE_LABELS` list in `build_scene_graph.py`. It contains *no* sizes, positions, or offsets. `IMMOVABLE_LABELS_GZ` similarly lists which categories count as furniture (movable objects connect to the nearest one of these). `<include>` names not present in `NAME_TO_LABEL` – walls, doors, door stoppers, the lawn/garden mesh, etc. – are structural and skipped.

<code><include><name></code>	<code>model:// URI</code>	Label	Kind
wagon	wagon	trolley	furniture
nasal_cupsul	nasal_inflammation_capsul	medicine box	object
sofa01, sofa02	sofa-fix	couch	furniture
kitchen_lowtable	kitchen_lowtable	coffee table	furniture
office_desk01/02/06	office_desk	desk	furniture
chair00/01/02/05	kitchen_chair	chair	furniture
living_sideboard	openable_living_sideboard	cabinet	furniture
wallet	wallet	wallet	object
high_table01	high_table	table	furniture
high_shelf01/02/03	high_shelf	shelf	furniture
hsr_orange_02	hsr_orange	orange	object

<code><include><name></code>	<code>model:// URI</code>	Label	Kind
<code>hsr_pringles_01/02</code>	<code>hsr_pringles</code>	pringles can	object
<code>apple_01</code>	<code>apple</code>	apple	object
<code>banana_01</code>	<code>banana</code>	banana	object
<code>pear_01</code>	<code>pear</code>	pear	object

Table 1: NAME_TO_LABEL registry: 24 `<include>` instances, 15 unique `model://` URIs. “Kind” is determined by membership in `IMMOVABLE_LABELS_GZ`.

4.2 Geometry extraction from Gazebo SDF

4.2.1 Motivation

An earlier version of this builder paired each label with a hand-picked (`width`, `depth`, `height`, `center_z`) tuple (a module-level `LABEL_GEOMETRY` dictionary). Those numbers were rough guesses and were shared across every instance of a label, so e.g. all four kitchen chairs used the same guessed size regardless of the model actually placed in the world. `core/perception/scene_graph/gazebo_geometry.py` replaces this entirely: it loads each model’s own SDF file and computes its bounding box from the **collision geometry** Gazebo itself uses for physics – the same shapes the simulator would use to detect the robot bumping into the object.

4.2.2 SDF frame hierarchy and pose composition

A Gazebo `model://` reference resolves to a directory containing `model.config` (which names the actual `.sdf` file) and that SDF file’s `<model>` element, which contains one or more `<link>` elements, each containing one or more `<collision>` (or `<visual>`) elements, each with a `<geometry>`. Every level may carry its own `<pose>x y z roll pitch yaw</pose>`, defaulting to identity if absent. Figure 2 shows how these poses compose to bring a shape’s local geometry into the model frame, and how the world-file `<include><pose>` (Section 4.1) then brings the model frame into the world frame.

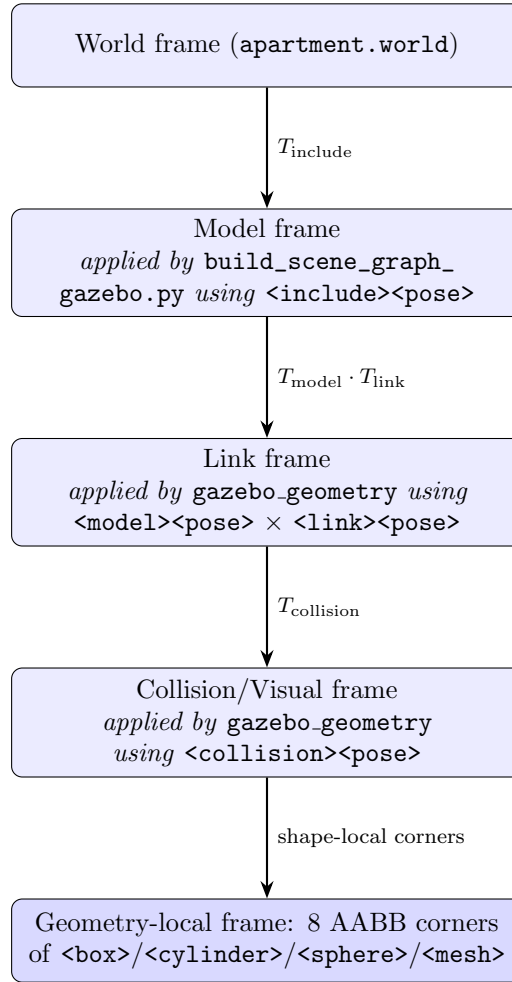


Figure 2: Pose composition chain. Every T is built by `pose_to_matrix()` from an SDF `<pose>x y z roll pitch yaw</pose>` string (identity if the element is absent). The bottom three levels (model, link, collision) are composed by `gazebo_geometry.model_local_bbox()` to give a single bounding box in the *model* frame; `build_scene_graph_gazebo.py` then applies T_{include} (the world-file pose) when sampling points (Section 4.3).

All rotations use the SDF convention $R = R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$ and $p_{\text{world}} = R p_{\text{local}} + t$, implemented once and shared by both the geometry module and the point synthesizer:

Listing 2: `gazebo_geometry.py`: shared rotation and pose-matrix helpers

```

1 def euler_to_matrix(roll, pitch, yaw):
2     cr, sr = np.cos(roll), np.sin(roll)
3     cp, sp = np.cos(pitch), np.sin(pitch)
4     cy, sy = np.cos(yaw), np.sin(yaw)
5     Rx = np.array([[1, 0, 0], [0, cr, -sr], [0, sr, cr]])
6     Ry = np.array([[cp, 0, sp], [0, 1, 0], [-sp, 0, cp]])
7     Rz = np.array([[cy, -sy, 0], [sy, cy, 0], [0, 0, 1]])
8     return Rz @ Ry @ Rx
9
10 def pose_to_matrix(pose_text):
11     T = np.eye(4)
12     if pose_text is None:
13         return T # default pose = identity
14     x, y, z, roll, pitch, yaw = (float(v) for v in pose_text.split())
  
```



```

15 T[:3, :3] = euler_to_matrix(roll, pitch, yaw)
16 T[:3, 3] = [x, y, z]
17 return T

```

4.2.3 Resolving model:// URIs

GAZEBO_MODEL_PATH (set by the ROS 2 workspace setup scripts) lists directories containing model packages. `resolve_uri()` searches it for a directory matching the URI's model name, and `_model_sdf_path()` reads that model's `model.config` to find the actual `.sdf` filename (SDF format versions are mapped to filenames there, e.g. `model-1.4.sdf`):

Listing 3: `gazebo_geometry.py`: URI resolution (excerpt)

```

1 def resolve_uri(uri):
2     if not uri.startswith("model://"):
3         return Path(uri)
4     name, _, sub = uri[len("model://")].partition("/")
5     for base in _gazebo_model_paths(): # GAZEBO_MODEL_PATH entries
6         candidate = base / name
7         if candidate.is_dir():
8             return candidate / sub if sub else candidate
9     raise FileNotFoundError(f"Could not resolve '{uri}' via GAZEBO_MODEL_PATH")
10
11 def _model_sdf_path(model_dir):
12     config = ET.parse(model_dir / "model.config").getroot()
13     return model_dir / config.findtext("sdf")

```

4.2.4 Collision shape primitives

For each `<collision>` (falling back to `<visual>` if a link has none), `_shape_corners()` dispatches on the geometry type and returns the 8 corners of that shape's axis-aligned bounding box in its *own* local frame:

- **box** – `<size>w d h</size>` → corners at $(\pm w/2, \pm d/2, \pm h/2)$.
- **cylinder** – `<radius>`, `<length>` (axis along local *Z*, per SDF convention) → corners of a $(\pm r, \pm r, \pm \text{length}/2)$ box.
- **sphere** – `<radius>` → corners of a $(\pm r, \pm r, \pm r)$ cube (a safe enclosing box for a sphere).
- **mesh** – see below.

Listing 4: `gazebo_geometry.py`: primitive corner helpers

```

1 _CORNER_SIGNS = np.array(list(itertools.product([-1, 1], repeat=3)))
2
3 def _box_corners(size_text):
4     half_extent = np.array([float(v) for v in size_text.split()]) / 2.0
5     return _CORNER_SIGNS * half_extent
6
7 def _cylinder_corners(geom):
8     r = float(geom.findtext("radius"))
9     length = float(geom.findtext("length"))
10    return _CORNER_SIGNS * np.array([r, r, length / 2.0])
11
12 def _sphere_corners(geom):
13     r = float(geom.findtext("radius"))
14     return _CORNER_SIGNS * r

```

4.2.5 Mesh shapes and the COLLADA unit pitfall

Several models (the sofa, kitchen chairs, the table legs, the wallet, the banana) use a `<mesh><uri>... .dae / .stl</uri>` collision shape instead of a primitive. These are loaded with `trimesh` and their axis-aligned bounds are used directly. There is one important subtlety: **COLLADA (.dae) files declare their own unit of length** in an `<asset><unit meter="X" .../></asset>` tag, and `trimesh` does *not* apply this automatically.

While building this module, `sofa-fix/meshes/sofa_seat.dae` was found to declare `<unit meter="0.1" name="decimeter"/>`. Loading it naively gave an absurd $7.76\text{ m} \times 12.57\text{ m} \times 4.76\text{ m}$ bounding box for a sofa seat. `_collada_unit_meters()` parses this tag directly from the XML (defaulting to 1.0 if absent) and the loaded mesh is scaled by it before computing bounds – after which the sofa seat mesh correctly measures $0.776\text{ m} \times 1.257\text{ m} \times 0.476\text{ m}$:

Listing 5: `gazebo_geometry.py`: mesh bounds with COLLADA unit correction

```
1 COLLADA_NS = {"c": "http://www.collada.org/2005/11/COLLADASchema"}
2
3 def _collada_unit_meters(dae_path):
4     unit = ET.parse(dae_path).getroot().find("./c:asset/c:unit", COLLADA_NS)
5     return float(unit.get("meter", "1.0")) if unit is not None else 1.0
6
7 @lru_cache(maxsize=None)
8 def _mesh_local_bounds(mesh_path):
9     path = Path(mesh_path)
10    mesh = trimesh.load(str(path), force="mesh")
11    if path.suffix.lower() == ".dae":
12        mesh.apply_scale(_collada_unit_meters(path))
13    return mesh.bounds[0].copy(), mesh.bounds[1].copy()
14
15 def _mesh_corners(geom):
16    path = resolve_uri(geom.findtext("uri"))
17    scale = np.array([float(v) for v in (geom.findtext("scale") or "1 1 1").split()])
18    bmin, bmax = _mesh_local_bounds(str(path))
19    bmin, bmax = bmin * scale, bmax * scale
20    return np.array(list(itertools.product(*zip(bmin, bmax))))
```

After the unit correction, the SDF's own `<scale sx sy sz>` (default 1, 1, 1) is applied on top – this is the scale Gazebo itself applies when instantiating the mesh as a collision shape, so the two corrections are independent and both necessary.

4.2.6 Assembling a model's bounding box

`model_local_bbox(uri)` is the single entry point used by the scene graph builder. For every `<link>` and every `<collision>/<visual>` shape inside it, the shape's local corners are transformed through $T_{\text{collision}}$, then T_{link} , then T_{model} (Figure 2); all resulting corners across the whole model are pooled, and the union's axis-aligned bounding box gives `dims` (extent) and `center` (offset of the box's centre from the model origin):

Listing 6: `gazebo_geometry.py`: `model_local_bbox` – the core algorithm

```
1 @lru_cache(maxsize=None)
2 def model_local_bbox(model_uri):
3     model_dir = resolve_uri(model_uri)
4     model = ET.parse(_model_sdf_path(model_dir)).getroot().find("model")
5     model_pose = pose_to_matrix(model.findtext("pose"))
6
7     corners = []
8     for link in model.findall("link"):
```

```

9     link_pose = model_pose @ pose_to_matrix(link.findtext("pose"))
10    for shape in link.findall("collision") or link.findall("visual"):
11        local_corners = _shape_corners(shape.find("geometry"))
12        if local_corners is None:
13            continue
14        shape_pose = link_pose @ pose_to_matrix(shape.findtext("pose"))
15        corners.append(local_corners @ shape_pose[:3, :3].T + shape_pose[:3, 3])
16
17    points = np.concatenate(corners, axis=0)
18    bbox_min, bbox_max = points.min(axis=0), points.max(axis=0)
19    return bbox_max - bbox_min, (bbox_min + bbox_max) / 2.0

```

The `@lru.cache` means each unique `model:// URI` is parsed (and any meshes loaded) exactly once, no matter how many `<include>`s reference it – e.g. the four `kitchen_chair` instances, three `high_shelf` instances, three `office_desk` instances, and two `sofa-fix / hsr_pringles` instances each share one computation. Figure 3 shows this as a flowchart.

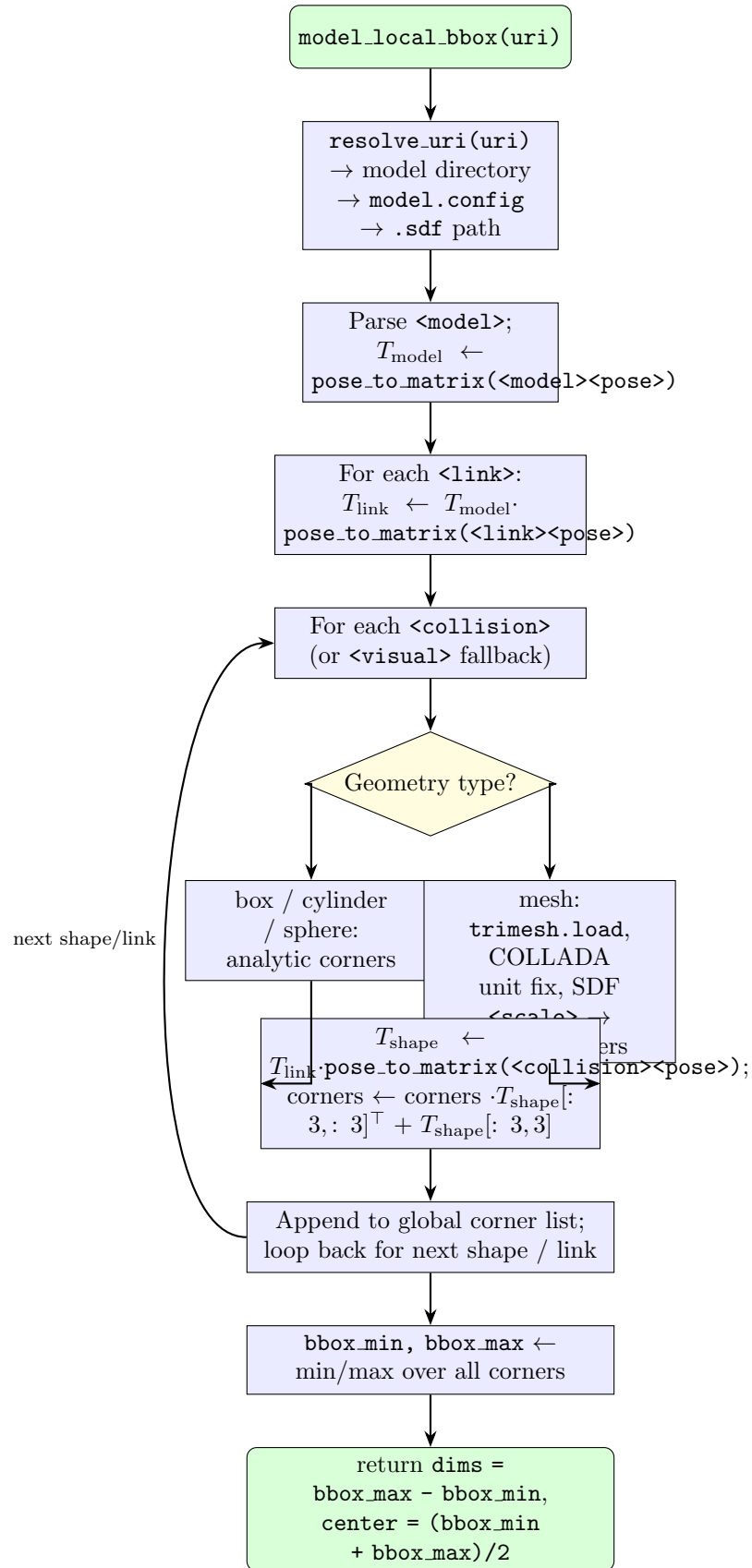


Figure 3: `gazebo_geometry.model_local_bbox()`: from a `model://` URI to a local-frame bounding box (`dims`, `center`), used by every `<include>` referencing that model.

4.3 Point cloud synthesis

Back in `build_scene_graph_gazebo.py`, `synthesize_points()` takes the `(dims, center)` pair from `model_local.bbox` and the `<include><pose>` from the world file, and produces a small $5 \times 5 \times 5$ grid of points filling that box, rotated and translated into the world frame – exactly the kind of point set the Mask3D pipeline would hand to `SceneGraph.add_node()` for a real scan:

Listing 7: `build_scene_graph_gazebo.py: synthesize_points`

```
1 def synthesize_points(pose, dims, center, n=5):
2     x, y, z, roll, pitch, yaw = pose
3     w, d, h = dims
4     cx, cy, cz = center
5
6     xs = cx + np.linspace(-w / 2, w / 2, n)
7     ys = cy + np.linspace(-d / 2, d / 2, n)
8     zs = cz + np.linspace(-h / 2, h / 2, n)
9     grid = np.stack(np.meshgrid(xs, ys, zs, indexing="ij"), axis=-1).reshape(-1, 3)
10
11     R = euler_to_matrix(roll, pitch, yaw)
12     return grid @ R.T + np.array([x, y, z])
```

4.4 ObjectNode: centroid, pose, and dimensions

Whether points come from `synthesize_points` (Gazebo) or from a Mask3D instance mask (real scan), they are handed to `core/perception/scene_graph/graph_nodes.py`'s `ObjectNode`, which computes everything the rest of the system needs:

Listing 8: `graph_nodes.py: centroid, PCA pose, and minimal OBB dimensions`

```
1 def __init__(self, object_id, color, sem_label, points, mesh_mask, confidence=None, movable=True):
2     ...
3     self.centroid = np.mean(points, axis=0)
4     ...
5     self.update_hull_tree()
6     self.compute_pose(self.points, self.centroid)
7     self.get_dimensions()
8
9 def compute_pose(self, points, centroid):
10     points_centered = points - centroid
11     covariance_matrix = np.cov(points_centered, rowvar=False)
12     eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
13     sorted_idx = np.argsort(eigenvalues)[::-1]
14     eigenvectors = eigenvectors[:, sorted_idx]
15     R = eigenvectors
16     if np.linalg.det(R) < 0:
17         R[:, -1] *= -1
18     object_pose = np.eye(4)
19     object_pose[:3, :3] = R
20     object_pose[:3, 3] = centroid
21     self.pose = object_pose
22
23 def get_dimensions(self):
24     point_cloud = o3d.geometry.PointCloud()
25     point_cloud.points = o3d.utility.Vector3dVector(self.points)
26     obb = point_cloud.get_minimal_oriented_bounding_box()
27     height_idx = np.argmax(np.abs(obb.R.T @ [0, 0, 1]))
28     width_depth = sorted([i for i in range(3) if i != height_idx],
29                           key=lambda i: obb.extent[i], reverse=True)
```

```

30 order = width_depth + [height_idx]
31 self.bb = obb
32 self.dimensions = obb.extent[order]

```

Two consequences are worth noting for interpreting the output table (Table 2):

- **Centroid** is the mean of the synthesized grid, i.e. $\text{pose}_{xyz} + R(\text{roll}, \text{pitch}, \text{yaw}) \cdot \text{center}$ – the real world-frame position of the geometric centre of the object’s collision geometry.
- **Dimensions** come from Open3D’s *minimal oriented bounding box* of the (rotated) point grid, then reordered so the axis most aligned with world Z is reported last. For an object rotated by 90° about Z (e.g. a desk with $\text{yaw}=-1.5708$), the reported width and depth are swapped relative to the model’s own (w, d, h) – this is expected and is the same convention `build_scene_graph.py` uses for real Mask3D instances.

4.5 SceneGraph: connections and persistence

`core/perception/scene_graph/scene_graph.py`’s `SceneGraph` class is shared, unchanged, by both builders. `add_node()` marks a node immovable if its label is in the builder’s `immovable` list; `update_connection()` gives every *movable* node one outgoing edge to its nearest *immovable* node (by centroid distance) – e.g. the apple connects to the high shelf it sits on:

Listing 9: `scene_graph.py`: `add_node` and `update_connection`

```

1 def add_node(self, color, sem_label, points, mesh_mask, confidence, movable=True):
2     if self.label_mapping.get(sem_label, "ID not found") in self.immovable:
3         self.nodes[self.index] = ObjectNode(self.index, np.array([0.5,0.5,0.5]),
4                                             sem_label, points, mesh_mask, confidence, movable=False)
5     else:
6         self.nodes[self.index] = ObjectNode(self.index, color, sem_label,
7                                             points, mesh_mask, confidence, movable=movable)
8     self.labels.setdefault(sem_label, []).append(self.index)
9     self.ids.append(self.index)
10    self.index += 1
11
12 def update_connection(self, node):
13     min_index, min_dist = None, None
14     if node.movable:
15         for other in self.nodes.values():
16             if not other.movable:
17                 dist = np.linalg.norm(node.centroid - other.centroid)
18                 if min_dist is None or dist < min_dist:
19                     min_dist, min_index = dist, other.object_id
20 ... # record one outgoing edge node -> min_index

```

After all nodes are added, `build_from_world()` (Gazebo) / `build_from_mask3d()` (real scan) both call `update_connection` for every node, rebuild a `scipy.spatial.KDTree` over centroids, assign colours via `color_with_ibm_palette()`, and call `scene_graph.save_all(graph_dir)`, which writes:

- `graph.json` – node ids/labels, the connections map (movable id $\rightarrow$ furniture id), and the list of immovable ids/labels;
- `scene.json` – `{furniture_id: {label, centroid, dimensions}}` for every immovable node;
- `objects/{id}.json` – one file per movable object with id, label, centroid, dimensions, PCA pose, and confidence.

4.6 End-to-end build flow

Figure 4 ties Sections 4.1–4.5 together as the top-level flow of `build_scene_graph_gazebo.py`'s `main()` / `build_from_world()`.

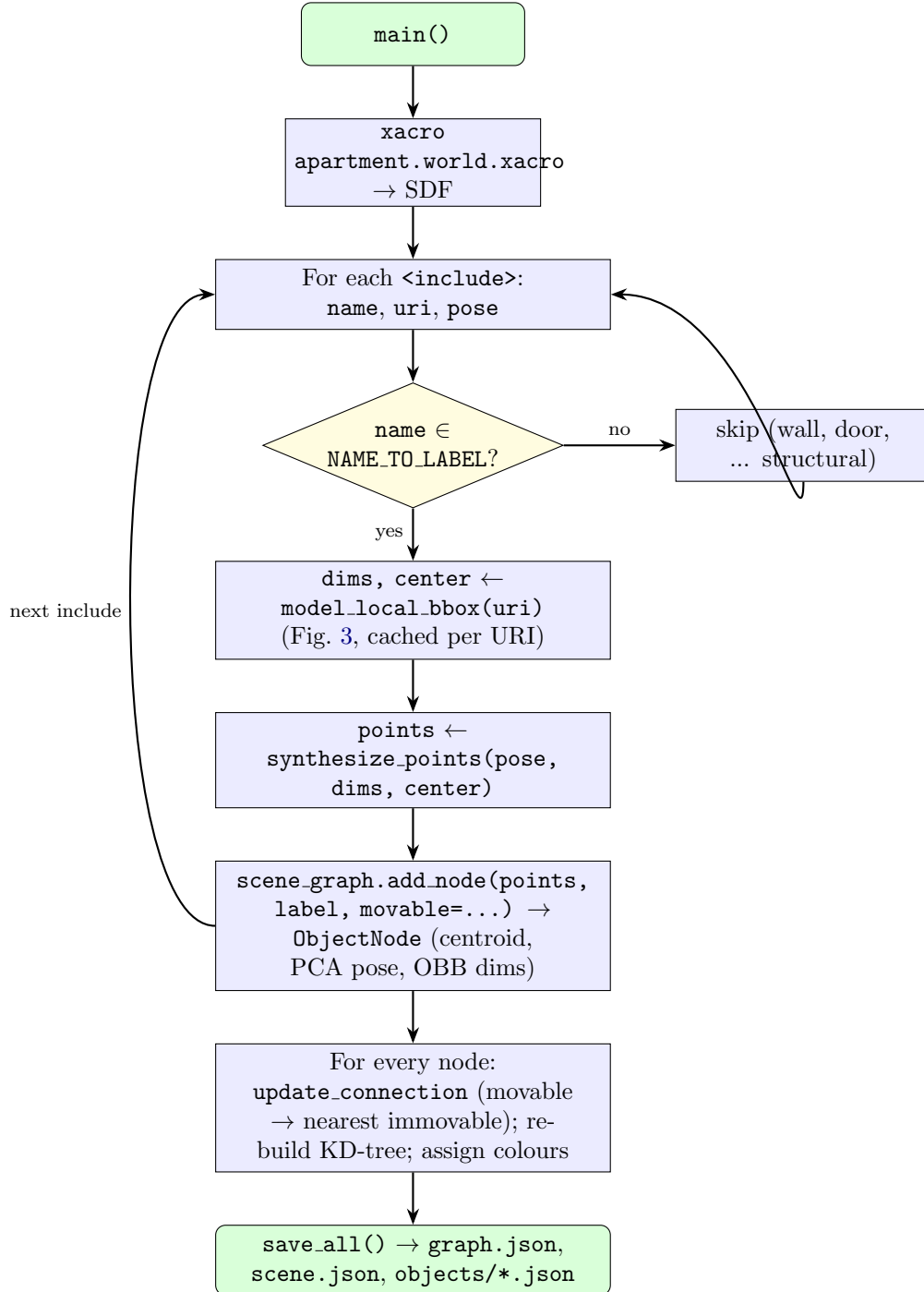


Figure 4: End-to-end flow of `build_scene_graph_gazebo.py`. Every box involving geometry (`model_local_bbox`, `synthesize_points`, `ObjectNode`) reads from Gazebo's own files or from the world-file pose – nothing is hand-picked.

4.7 Resulting scene graph

Running `python -m core.perception.scene_graph.build_scene_graph_gazebo` produces 24 nodes (16 furniture + 8 movable objects). Table 2 lists every node’s label, kind, world-frame centroid, and dimensions, all now derived purely from `apartment.world.xacro` poses and each model’s own SDF collision geometry.

id	label	kind	centroid (m)	dims $w \times d \times h$ (m)
0	trolley	furniture	(4.39, 1.42, 0.33)	$0.47 \times 0.44 \times 0.67$
1	medicine box	object	(4.50, 1.38, 0.69)	$0.13 \times 0.08 \times 0.03$
2	couch	furniture	(1.56, 1.66, 0.30)	$1.26 \times 0.78 \times 0.59$
3	couch	furniture	(1.56, 3.62, 0.30)	$1.26 \times 0.78 \times 0.59$
4	coffee table	furniture	(1.56, 2.58, 0.20)	$1.25 \times 0.80 \times 0.40$
5	desk	furniture	(9.10, 2.80, 0.35)	$1.20 \times 0.60 \times 0.70$
6	desk	furniture	(7.80, 2.80, 0.35)	$1.20 \times 0.60 \times 0.70$
7	desk	furniture	(7.80, 6.00, 0.35)	$1.20 \times 0.60 \times 0.70$
8	chair	furniture	(6.02, 3.04, 0.42)	$0.53 \times 0.52 \times 0.85$
9	chair	furniture	(7.90, 3.04, 0.42)	$0.53 \times 0.52 \times 0.85$
10	chair	furniture	(9.20, 3.04, 0.42)	$0.53 \times 0.52 \times 0.85$
11	chair	furniture	(7.90, 6.34, 0.42)	$0.53 \times 0.52 \times 0.85$
12	cabinet	furniture	(1.33, 0.27, 0.31)	$1.15 \times 0.48 \times 0.62$
13	wallet	object	(0.99, 0.27, 0.51)	$0.20 \times 0.10 \times 0.03$
14	table	furniture	(4.60, 4.00, 0.52)	$2.30 \times 0.50 \times 1.05$
15	shelf	furniture	(4.68, 13.02, 0.90)	$0.45 \times 0.30 \times 1.80$
16	shelf	furniture	(4.23, 13.02, 0.90)	$0.45 \times 0.30 \times 1.80$
17	shelf	furniture	(3.78, 13.02, 0.90)	$0.45 \times 0.29 \times 1.80$
18	orange	object	(4.57, 3.56, 1.11)	$0.07 \times 0.07 \times 0.12$
19	pringles can	object	(4.34, 13.00, 0.77)	$0.07 \times 0.07 \times 0.22$
20	pringles can	object	(4.67, 3.89, 1.16)	$0.07 \times 0.07 \times 0.23$
21	apple	object	(4.68, 13.05, 1.27)	$0.09 \times 0.09 \times 0.09$
22	banana	object	(3.75, 12.98, 0.38)	$0.13 \times 0.04 \times 0.02$
23	pear	object	(3.81, 13.03, 1.28)	$0.11 \times 0.11 \times 0.11$

Table 2: All 24 scene-graph nodes from `apartment.world`, with centroids and dimensions computed entirely from Gazebo SDF data (rounded to 2 decimals).

A few values are worth checking against intuition as evidence the geometry pipeline is correct: the apple’s 0.09 m diameter matches its `<sphere><radius>0.045</radius></sphere>` collision exactly; the wallet’s $0.20 \times 0.10 \times 0.03$ m box matches the `wallet.dae` mesh bounds; and the couch height of 0.59 m is the union of the (unit-corrected) sofa-seat mesh and its four 0.13 m-tall cylindrical legs – all read from `ros2_ws/install/tmc_gazebo_worlds/share/tmc_gazebo_worlds/models/`, none typed in by hand.

4.8 The real-scan pipeline (Mask3D)

`core/perception/scene_graph/build_scene_graph.py` builds the same `SceneGraph` from a real RGB-D scan, after `docker/mask3d/run_mask3d.sh` has produced `predictions.txt` and per-instance point

masks. core/perception/segmentation/mask3d_interface.py parses that output against the ScanNet200 label set (core/utils/scannet_200_labels.py):

Listing 10: mask3d_interface.py: loading Mask3D predictions

```

1 ID_TO_LABEL = dict(zip(VALID_CLASS_IDS_200, CLASS_LABELS_200))
2 LABEL_TO_ID = dict(zip(CLASS_LABELS_200, VALID_CLASS_IDS_200))
3
4 @dataclass
5 class Mask3DInstance:
6     mask_file: Path
7     class_id: int
8     label: str
9     confidence: float
10
11     def load_mask(self):
12         return np.loadtxt(self.mask_file, dtype=np.int64).astype(bool)

```

Because Mask3D’s per-instance masks can overlap, `build_from_mask3d()` assigns each point to exactly one instance – larger masks are processed first so that small objects resting on furniture override the furniture’s mask for their points (mirroring stretch-compose’s `scenegraph_preprocessing.py`):

Listing 11: build_scene_graph.py: one-point-one-owner mask resolution

```

1 order = np.argsort([-m.sum() for m in masks]) # largest masks first
2 owner = np.full(n_pts, -1, dtype=np.int64)
3 for inst_idx in order:
4     owner[masks[inst_idx]] = inst_idx
5
6 for inst_idx, inst in enumerate(instances):
7     mesh_mask = owner == inst_idx
8     node_points = np_points[mesh_mask]
9     if node_points.shape[0] < min_points:
10         continue
11     scene_graph.add_node(color=np_colors[mesh_mask][0], sem_label=inst.class_id,
12                          points=node_points, mesh_mask=mesh_mask,
13                          confidence=inst.confidence)

```

IMMOVABLE_LABELS (ScanNet200 furniture categories) and IGNORE_LABELS (walls, floors, doors, ...) play exactly the roles of IMMOVABLE_LABELS_GZ and “skip if not in NAME_TO_LABEL” in the Gazebo builder. From this point on – ObjectNode construction, connections, JSON export, room clustering, and scene queries – both pipelines are identical (Figure 1).

5 Demo-Day Requirement to Code Correlation

This section walks through the ten demo-day requirements from Section 2’s introduction one at a time and points to the exact file and function that implements each one. Table 3 is the quick-reference version; the rest of the section explains each row in detail, with the relevant code excerpts.

#	Requirement	Implementing code	See
1	3D scene graph from Gazebo ground truth	build_scene_graph_gazebo.py, gazebo_geometry.py, scene_graph.py, graph_nodes.py	§4
2	LLM clusters furniture into rooms	core/llm_zone/room_clustering.py	§5.1

#	Requirement	Implementing code	See
3	Scenario A / B target resolution	<code>scene_query.query()</code> , <code>scene_query.predict_locations()</code> , <code>search_and_fetch.run()</code>	§5.2
4	Navigate to the first candidate location	<code>active_perception.approach_pose()</code> , <code>HsrRobotAdapter.navigate_to()</code>	§5.3
5	Observe location; detect or not	<code>search_and_fetch._observe_and_detect()</code> , <code>HsrRobotAdapter.look_at()/get_rgbd()</code> , <code>Sam3Client.detect()</code>	§5.4
6	If detected: point cloud, grasp, arm move	<code>mask_to_pointcloud()</code> , <code>grasp_planner.compute_grasp_pose()</code> , <code>HsrRobotAdapter.grasp()</code>	§5.5
7	If not detected: next location, then LLM fallback	<code>search_and_fetch._search_location()</code> , <code>run()</code> candidate queue	§5.6
8	Active perception: head control	<code>head_geometry.py</code> , <code>active_perception.viewpoints()</code> , <code>HsrRobotAdapter.look_at()</code>	§5.7
9	Scene graph from a real RGB-D scan (Mask3D)	<code>build_scene_graph.py</code> , <code>mask3d_interface.py</code>	§4.8
10	Steps 3–7 as a single command	<code>ros2 run fetcher search_and_fetch</code> , <code>search_and_fetch_node.py</code>	§5.8

Table 3: Demo-day requirements mapped to the code that implements them.

Requirements 1 and 9: scene graph construction

Both the Gazebo and the real-scan scene graphs are built by Section 4: `build_scene_graph_gazebo.py` + `gazebo_geometry.py` (§4.1–§4.2) for requirement 1, and `build_scene_graph.py` + `mask3d_interface.py` (§4.8) for requirement 9. Both converge on the same `SceneGraph/ObjectNode` representation and JSON files (§4.5), so everything described below works identically regardless of which one produced the scene graph.

5.1 Requirement 2: room clustering with an LLM

`core/llm_zone/room_clustering.py` reads `scene.json`’s furniture list (label + map-frame centroid, both produced in §4.7) and asks DeepSeek to group the furniture ids into named rooms. The system prompt instructs the model to use *both* semantics (“a stove and a refrigerator belong to a kitchen”) and spatial proximity:

Listing 12: `room_clustering.py`: the clustering prompt and centroid attachment (lines 31–40, 73–80)

```

1 SYSTEM = (
2     "You are a spatial reasoning agent. You receive a JSON list of furniture "
3     "items, each with: id (int), label (str), centroid [x, y, z] in metres "
4     "(map frame). Cluster them into rooms using both semantics (a stove and a "
5     "refrigerator belong to a kitchen) and spatial proximity (items in one "
6     "room are close together). Use common room names (kitchen, living room, "
7     "bedroom, office, bathroom, hallway). Every furniture id must appear in "
8     "exactly one room. Return ONLY valid JSON: "
9     '{"rooms": [{"name": str, "furniture_ids": [int, ...]}}'
```

```

10 )
11
12 # after the LLM call, attach a geometric centroid per room (not LLM-provided)
13 centroid_of = {f["id"]: np.array(f["centroid"]) for f in furniture}
14 for room in result.get("rooms", []):
15     ids = [i for i in room.get("furniture_ids", []) if i in centroid_of]
16     room["centroid"] = (
17         np.mean([centroid_of[i] for i in ids], axis=0).tolist() if ids else None
18     )

```

The LLM only ever sees ids, labels and centroids – it never invents coordinates. `main()` writes the result to `data/scene_graph/rooms.json`. This file is read later by `scene_query.predict_locations()` (§5.2) to add a room field to each furniture candidate, which helps the LLM reason about “the cup is probably in the kitchen” style predictions.

5.2 Requirement 3: Scenario A / B target resolution

`core/llm_zone/scene_query.py` is the bridge between a free-text target (e.g. “a cup”) and the scene graph. `load_scene()` turns `graph.json` + `scene.json` into a flat list of items (id, label, type, and – for movable objects – the furniture they are on, via the same `connections` edges built in §4.5), plus a centroid lookup where movable objects inherit the centroid of the furniture they sit on:

Listing 13: `scene_query.py`: `query()` attaches type and centroid from the local lookup, never from the LLM (lines 131–138)

```

1 oid = result.get("object_id")
2 if oid is not None:
3     matched = next((it for it in items if it["id"] == oid), None)
4     result["type"] = matched["type"] if matched else None
5     result["centroid"] = centroid_lookup.get(oid)
6 else:
7     result["type"] = None
8     result["centroid"] = None

```

`query()`’s system prompt asks DeepSeek-R1 to pick the single best matching item id for the user’s query (present, object_id, reasoning); the Python code then looks up that id’s type/centroid itself. `search_and_fetch.run()` uses the result to choose between the two scenarios from Section 2’s requirement list:

Listing 14: `search_and_fetch.py`: `run()` scenario selection (lines 96–108)

```

1 answer = scene_query.query(target)
2 candidates: list[dict] = []
3 if answer.get("present") and answer.get("centroid"):
4     # Scenario B: target already in the scene graph -> go straight there
5     candidates.append({
6         "centroid": answer["centroid"],
7         "furniture_label": answer.get("object_label", "known location"),
8     })
9 else:
10    # Scenario A: target not in the scene graph -> ask the LLM where to look
11    candidates = scene_query.predict_locations(target, k=k)

```

Scenario A (`present=False`, e.g. the target is a kind of object that was never placed in the world) calls `predict_locations()`, whose system prompt asks DeepSeek-Chat for the top-*k* furniture ids most likely to hold the object, enriched with the room names from `rooms.json` (§5.1). **Scenario B** (`present=True`) skips the LLM entirely for the first candidate and goes straight to the object’s (or

its supporting furniture’s) known centroid – this is the fast path for objects that are already part of the scene graph built in §4.

5.3 Requirement 4: navigating to the candidate location

Every candidate carries a map-frame centroid. `approach_pose()` turns that 3D point into a 2D Nav2 goal: a pose offset by a `standoff` distance (default 0.8m, the robot’s footprint plus margin) so the robot stops in front of the object rather than on top of it, oriented to face the target:

Listing 15: `active_perception.py: approach_pose()` (lines 22–41)

```
1 def approach_pose(centroid, standoff=0.8, face_target=True) -> dict:
2     x, y = centroid[0], centroid[1]
3     approach_x = x - standoff
4     approach_y = y
5     yaw = math.atan2(y - approach_y, x - approach_x) # pointing at target
6     qz = math.sin(yaw / 2)
7     qw = math.cos(yaw / 2)
8     return {"x": approach_x, "y": approach_y, "qz": qz, "qw": qw}
```

`HsrRobotAdapter.navigate_to()` (`robot_adapter.py`, lines 143–155) is the only place this pose dict is consumed: it is packed into a `geometry_msgs/PoseStamped` in the map frame and sent to `nav2_simple_commander.BasicNavigator.goToPose()`, the same Nav2 action client used by `good_boy.py`. The call blocks (polling `isTaskComplete()`) until Nav2 reports success or failure, and returns a plain `bool` – this is the only signature the pure-Python mission code in `search_and_fetch.py` needs to know about.

5.4 Requirement 5: observing the location and detecting the object

Once the robot is parked at the approach pose, `_observe_and_detect()` performs the actual “look and check” step:

Listing 16: `search_and_fetch.py: _observe_and_detect()` (lines 64–72)

```
1 def _observe_and_detect(robot, sam3, target, centroid):
2     """Look at the target location, run SAM3; return (mask, depth, K) or None."""
3     robot.look_at(centroid)
4     rgb, depth, K = robot.get_rgbd()
5     det = sam3.detect(rgb, target)
6     if det.found:
7         return det.best_mask(), depth, K
8     return None
```

`robot.look_at(centroid)` drives the head (detailed in §5.7); `robot.get_rgbd()` returns the latest synchronized colour image, depth image and camera intrinsics K from the head RGB-D camera topics. `sam3.detect()` (`core/perception/detection/sam3_client.py`, lines 70–97) POSTs the JPEG-encoded RGB frame and the text `target` to the SAM3 docker server’s `/sam3/predict` endpoint and decodes the response into a `Sam3Detection` (boolean instance masks, boxes, confidence scores). `det.found` is simply `masks.shape[0] > 0` – “detected” vs “not detected” is exactly this open-vocabulary SAM3 result, with no Gazebo-specific shortcuts.

5.5 Requirement 6: detected – point cloud, grasp, arm motion

If `det.found`, `_observe_and_detect()` returns `det.best_mask()` (the highest-confidence instance) together with the depth image and K . `run()` then performs the three remaining sub-steps of requirement 6:

Listing 17: `sam3_client.py: mask_to_pointcloud()` – 2D mask + depth $\rightarrow$ 3D points in the camera frame (lines 100–130)

```

1 def mask_to_pointcloud(mask, depth, intrinsics, depth_scale=1000.0):
2     fx, fy = intrinsics[0, 0], intrinsics[1, 1]
3     cx, cy = intrinsics[0, 2], intrinsics[1, 2]
4
5     v, u = np.where(mask)
6     z = depth[v, u].astype(np.float64) / depth_scale
7     valid = z > 0
8     u, v, z = u[valid], v[valid], z[valid]
9
10    x = (u - cx) * z / fx
11    y = (v - cy) * z / fy
12    pts = np.stack([x, y, z], axis=1)
13
14    pcd = o3d.geometry.PointCloud()
15    pcd.points = o3d.utility.Vector3dVector(pts)
16    return pcd

```

This is the standard pinhole back-projection $X = (u - c_x)z/f_x$, $Y = (v - c_y)z/f_y$, $Z = z$, restricted to the pixels where `mask` is `True` and depth is valid ($z > 0$). The resulting `object_pcd` is an Open3D point cloud in the head camera’s optical frame.

`HsrRobotAdapter.grasp(object_pcd)` (`robot_adapter.py`, lines 223–282) is the ROS 2 side of “plan a grasp and move the arm”. It first calls `grasp_planner.compute_grasp_pose()`:

Listing 18: `grasp_planner.py: top-down grasp from the object point cloud’s centroid` (lines 12–29)

```

1 TOP_DOWN_QUAT = (1.0, 0.0, 0.0, 0.0) # hand pointing straight down
2 PREGRASP_HEIGHT = 0.05 # metres above the centroid
3
4 def compute_grasp_pose(object_pcd) -> dict:
5     centroid = np.asarray(object_pcd.points).mean(axis=0)
6     return {
7         "position_camera": centroid.tolist(),
8         "orientation": TOP_DOWN_QUAT,
9         "pregrasp_height": PREGRASP_HEIGHT,
10    }

```

`grasp()` then: (1) transforms `position_camera` from the camera frame to `odom` via TF (`tf2_geometry_msgs`); (2) calls the `ik_solver_node/solve_ik_with_collision` service with the target hand pose (centroid + `pregrasp_height` in z , orientation `TOP_DOWN_QUAT`); (3) opens the gripper (`GRIPPER_OPEN= 1.2`), publishes the returned IK joint solution to `/arm_trajectory_controller/joint_trajectory`, closes the gripper (`GRIPPER_CLOSE= -0.6`), and finally raises `arm_lift_joint` by `LIFT_DELTA= 0.05m` as a visible “I picked it up” flourish. Each step is followed by a `spin_for(...)` settle delay so the next sensor/IK call sees the robot’s new state. The function returns `True/False`, which becomes `MissionResult.grasped`.

5.6 Requirement 7: not detected – next location, then LLM fallback

If `_observe_and_detect()` returns `None` at the approach pose, `_search_location()` does not give up immediately – it tries a few orbit viewpoints first (this is also requirement 8’s active-perception search):

Listing 19: `search_and_fetch.py: _search_location()` (lines 75–89)

```

1 def _search_location(robot, sam3, target, centroid, n_viewpoints=3):
2     if not robot.navigate_to(approach_pose(centroid)):
3         return None
4     hit = _observe_and_detect(robot, sam3, target, centroid)

```

```

5  if hit:
6      return hit
7  for pose in viewpoints(centroid, n=n_viewpoints):
8      if not robot.navigate_to(pose):
9          continue
10     hit = _observe_and_detect(robot, sam3, target, centroid)
11     if hit:
12         return hit
13 return None

```

If *that* whole location comes up empty, `run()`'s outer loop moves on to the next candidate in its queue. The fallback to the LLM happens once that queue is empty – this covers both the end of Scenario A's k candidates and Scenario B's single known-location guess:

Listing 20: `search_and_fetch.py: run()` – LLM fallback once the candidate queue is empty (lines 131–142)

```

1  print(f"[mission] Step 7: '{target}' not at '{label}'")
2  if not candidates:
3      # Scenario B fallback (and scenario A exhaustion): ask the LLM,
4      # skipping locations we already visited
5      more = scene_query.predict_locations(target, k=k)
6      candidates = [
7          m for m in more
8          if m.get("centroid")
9          and tuple(np.round(m["centroid"], 2)) not in visited
10 ]

```

The full outer loop – including the `visited` set that prevents re-trying the same centroid – is diagrammed in Figure 5 (Section 6).

5.7 Requirement 8: active perception (head control)

Two layers implement active perception. `core/perception/head_geometry.py` is pure trigonometry, with no ROS dependency:

Listing 21: `head_geometry.py: pan/tilt angles from a 3D target` (lines 24–44)

```

1  PAN_LIMITS = (-3.84, 1.75)
2  TILT_LIMITS = (-1.57, 0.52)
3
4  def pan_delta(x_in_pan_frame, y_in_pan_frame) -> float:
5      """Additional head_pan_joint rotation so +X of the pan link points at (x, y)."""
6      return math.atan2(y_in_pan_frame, x_in_pan_frame)
7
8  def tilt_for_target(x_base, y_base, z_base, head_height) -> float:
9      """head_tilt_joint angle to point the camera at a base-frame target."""
10     horiz = math.hypot(x_base, y_base)
11     return math.atan2(z_base - head_height, max(horiz, 1e-3))

```

`HsrRobotAdapter.look_at(point_map)` (`robot_adapter.py`, lines 159–192) is the glue: it TF-transforms the target point into the current `head_pan_link` frame to get `pan_delta`, looks up `head_pan_link`'s height above `base_footprint` for `tilt_for_target`, clamps both angles to `PAN_LIMITS` / `TILT_LIMITS`, and publishes a two-joint `JointTrajectory` to `/head_trajectory_controller/joint_trajectory`. This is called once per observation in `_observe_and_detect()` (§5.4).

The *search* part of active perception – where to look next if the first view doesn't show the object – is `active_perception.viewpoints()`: it yields n poses (default 6) evenly spaced on a circle of radius metres (default 0.9 m) around the target centroid, each oriented to face the centroid, so the

robot orbits the object looking for an unoccluded view. `_search_location()` (§5.6) drives Nav2 to each of these poses and re-runs `_observe_and_detect()` – i.e. “navigate, look with the head, detect” repeated from several angles, which *is* the active-perception demonstration. (`active_perception.py` also exposes a standalone `active_search()` helper with the same loop, for callers that only have `is_visible/navigate_to` callbacks and don’t go through SAM3.)

5.8 Requirement 10: a single command for steps 3–7

`ros2_ws/src/fetcher/setup.py` registers `search_and_fetch = fetcher.search_and_fetch_node:main` as a console script, so the entire mission is one command:

Listing 22: Running the mission

```
1 DEEPSEEK_API_KEY=<key> ros2 run fetcher search_and_fetch "<object>"
```

`search_and_fetch_node.py` is intentionally tiny – it only wires the ROS 2 `HsrRobotAdapter` (§5.3–§5.7) into the ROS-agnostic `run()` from §5.2–§5.6:

Listing 23: `search_and_fetch_node.py`: the entire node (lines 18–33)

```
1 def main():
2     target = " ".join(sys.argv[1:])
3
4     rclpy.init()
5     adapter = HsrRobotAdapter()
6     threading.Thread(target=rclpy.spin, args=(adapter,), daemon=True).start()
7     adapter.wait_until_ready()
8
9     result = run(target, adapter)
10    print(f"\n[search_and_fetch] {result}")
11
12    rclpy.shutdown()
```

`rclpy.spin` runs on a background thread so that `HsrRobotAdapter`’s subscriptions/TF buffer keep updating while `run()` (main thread) blocks on `navigate_to`, `look_at`, etc. – exactly the threading note in `robot_adapter.py`’s module docstring. `wait_until_ready()` blocks until the camera, joint states, TF and Nav2 are all live before `run()` starts, so step 4’s first `navigate_to` call never races a cold start.

6 Single-Command Mission: Detailed Flow

Section 5 walked through requirements 3–8 file by file. This section ties them together as two flowcharts: Figure 5 is `search_and_fetch.run()`’s outer loop over candidate locations (scenario selection, the visited set, and the LLM fallback), and Figure 6 is the per-location search loop (`_search_location` / `_observe_and_detect`) that Figure 5 calls for every candidate.

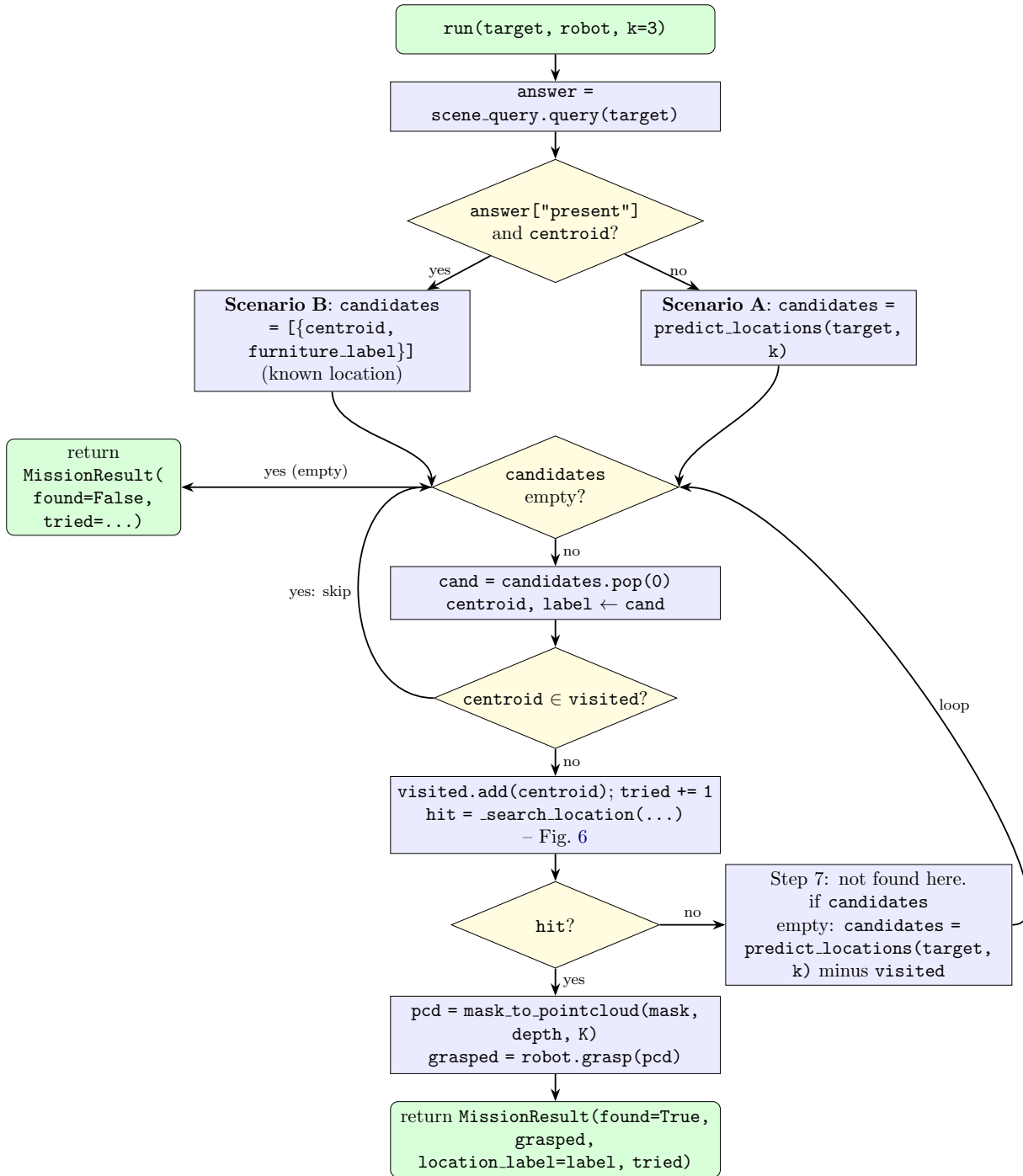


Figure 5: `search_and_fetch.run()`: scenario selection, the candidate/visited loop, and the LLM fallback once the queue is empty. The “`_search_location`” box is expanded in Figure 6.

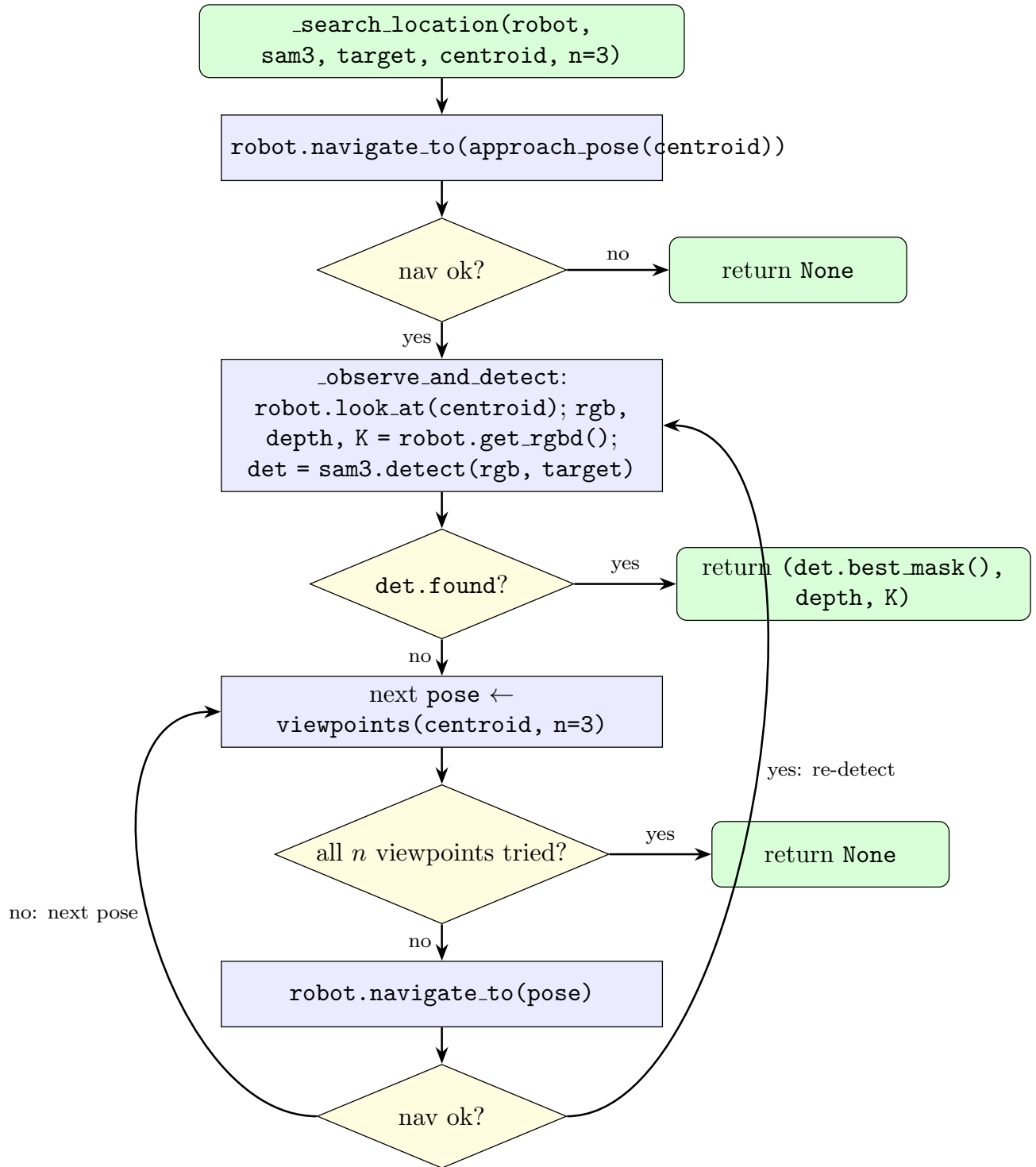


Figure 6: `_search_location()` / `_observe_and_detect()`: the per-location active-perception loop (requirements 5, 7 and 8). The robot first checks the direct approach pose, then orbits the target through up to `n_viewpoints` poses from `active_perception.viewpoints()`, re-running detection from each one, before giving up on this location.

7 Running the Pipeline

This section collects the commands used to run the whole system end to end, in the order they are normally executed. All commands run inside the devcontainer unless marked (*host*).

7.1 One-time setup

Listing 24: Mask3D setup (host, only needed for requirement 9)

```
1 bash docker/mask3d/setup_mask3d.sh
```

7.2 Bring up the simulation

Listing 25: Gazebo + Nav2 (Section 3)

```
1 ros2 launch hsrc_gazebo_launch hsrc_apartment_world.launch.py
2 ros2 launch hsrc_rosnav_config navigation_launch.py map:=/home/ws/map.yaml
```

Listing 26: SAM3 detection server (host, requirement 5)

```
1 export HF_TOKEN=hf_xxx # accept the facebook/sam3 license on Hugging Face first
2 bash docker/sam3/run_sam3.sh
```

Listing 27: IK solver (requirement 6)

```
1 ros2 launch fetcher ik_solver.launch.py
```

7.3 Build the scene graph and room clusters (once per scene)

Listing 28: Requirement 1: scene graph from Gazebo ground truth (Section 4)

```
1 ros2 run fetcher gazebo_scene_graph
2 # or, equivalently, without ROS:
3 python -m core.perception.scene_graph.build_scene_graph_gazebo [--visualize]
```

Listing 29: Requirement 9: scene graph from a real RGB-D scan (Section 4.8)

```
1 bash docker/mask3d/run_mask3d.sh [workspace] [mask3d_repo] # host
2 python -m core.perception.scene_graph.build_scene_graph [--workspace <path>] [--visualize]
```

Listing 30: Requirement 2: room clustering (Section 5.1)

```
1 DEEPSEEK_API_KEY=<key> python -m core.llm_zone.room_clustering
```

7.4 Run the mission (requirements 3–7, single command)

Listing 31: Requirement 10: single-command search and fetch (Section 5.8)

```
1 DEEPSEEK_API_KEY=<key> ros2 run fetcher search_and_fetch "<object>"
```

For ad-hoc testing of just the scene-reasoning layer (no robot), the DeepSeek-backed query can be run standalone:

Listing 32: Scene query only (Section 5.2)

```
1 DEEPSEEK_API_KEY=<key> python core/llm_zone/scene_query.py "something to drink"
```

8 Conclusion

Every demo-day requirement maps to a small, single-purpose module: Gazebo’s own SDF files drive scene-graph geometry (§4), DeepSeek clusters rooms and resolves search targets (§5.1, §5.2), and the navigate/observe/grasp loop (Figures 5–6) is shared verbatim between Scenario A and Scenario B. The same `SceneGraph` / `ObjectNode` representation, and the same JSON files, are produced whether the source is Gazebo’s ground truth (§4.1–§4.7) or a real Mask3D scan (§4.8), so the LLM and mission layers are unaware of which one was used. Most importantly for this report’s original motivation: nothing in §4 is hand-picked – every object’s position comes from `apartment.world.xacro`’s `<include><pose>`, and every object’s size and shape comes from parsing that same object’s own `model.sdf` collision geometry (§4.2), so adding, moving, or resizing an object in Gazebo automatically and correctly updates the scene graph with no code changes.